

robot-dev 机器人侧终端与执行说明

适用：TonyPi 机器人（`raspberrypi-9`）与开发 PC 的日常操作

整理日期：2026-09-21 | 仓库分支：`dxd_nine_grid` | 机器人代码目录：`/home/pi/Robot_Competition`

0. 一句话总览

一共三层“终端”，各管各的：

层	是什么	干什么
① PC 终端	PowerShell 7.6.6 (Windows)，工作目录 <code>F:\coding\Projects\RoboTrack\robot-dev</code>	跑 PC 侧脚本：测试、离线分析、以及“指挥机器人”的工具
② 机器人终端	ssh / 网页终端登入 <code>pi@raspberrypi-9</code> ，工作目录 <code>/home/pi/Robot_Competition</code>	跑关卡、跑现场标定/测量工具、启停 debug 服务
③ PC→机器人 远程层	PC 上用 Jupyter Contents API / 内核会话远程操作机器人	同步代码、远程执行、拉回数据（现场测量主要走这条）

1. 机器人侧终端的实测环境

项	值
主机 / 用户	<code>raspberrypi-9 / pi</code>
<code>/bin/sh</code>	<code>dash</code> （不是 <code>bash</code> ）—— 关键坑，见第 2 节
可用 shell	<code>bash</code> （ <code>bash --version</code> 正常）
我们的工作目录	<code>/home/pi/Robot_Competition</code>
系统	Debian GNU/Linux 13 (trixie), Python 3.13.5
Jupyter 服务	<code>jupyter.service</code> （自启）， <code>User=pi</code> ， <code>WorkingDirectory=/home/pi</code> ，监听 <code>8888</code> ，密码 <code>pi</code>
debug 镜像服务	端口 <code>8081</code> ，由 <code>bash debug.sh</code> 启停（不占用摄像头）

※ 本表为 2026-09-21 会话内实测值。撰写本 PDF 时 PC 处于 `Tsinghua-Secure` 网络、机器人不可达，故未能当场复测；恢复网络后建议复核一遍。

2. 第一条铁律：shell 脚本一律用 bash

<code>bash debug.sh</code>	# 正确
<code>./debug.sh</code>	# 也可以（shebang 是 <code>#!/usr/bin/env bash</code> ）
<code>sh debug.sh</code>	# <code>dash</code> 不支持 <code>set -u</code> → "Illegal option -"

典型报错（已实际踩到）：

```
: invalid option: set: -
debug.sh: line 5: cd: $'\r': No such file or directory
debug.sh: line 9: syntax error near unexpected token `'$'\r''
```

两半原因：

- ① `invalid option: set: -` → 用了 `sh` (dash)，不支持 `set -u`；
- ② 满屏 `\r` → 文件是 **CRLF 行尾**。Windows 上 `core.autocrlf=true` 会把工作树 checkout 成 CRLF，而同步工具是逐字节传文件，于是把 CRLF 带上了机器人。

已修： `tools/sync_to_robot.py` 现在对 `.sh / .bash` 上传前自动 CRLF→LF 并打印 [行尾] 提示。

3. Python 环境：两个解释器，必须选对

机器人上**只有一个 Python 二进制 (3.13.5)**，但有两个不同的“包世界”：

解释器	是否可用	包集合
<code>/home/pi/jupyter-env/bin/python3</code>	✅ 我们的代码必须用它	venv site-packages: <code>numpy 2.5.3</code> 、 <code>cv2 4.14.0</code> 、 <code>scipy</code> 、 <code>matplotlib</code> 、 <code>scikit-learn</code> 、 <code>scikit-image</code> 、 <code>joblib</code> 、 <code>onnxruntime</code> 、 <code>apriltag</code> 、 <code>yaml</code> 、 <code>serial</code> 、 <code>hiwonder</code>
<code>/usr/bin/python3</code>	❌ 不要用	Debian <code>dist-packages</code> ：有 <code>numpy 2.2.4</code> / <code>cv2</code> / <code>serial</code> / <code>smbus</code> / <code>RPi</code> ， 没有 <code>scipy</code> / <code>apriltag</code> / <code>onnxruntime</code> / <code>sklearn</code>

容易绕晕的一点： `/home/pi/jupyter-env/bin/python3` 是个软链，指向 `/usr/bin/python3` —— 同一个二进制。真正起作用的是 `pyvenv.cfg` (`include-system-site-packages=false`)，它切换了 `sys.path`。所以“看 `which python3`”分辨不出来，必须看路径前缀。

用法（二选一）

```
PY=/home/pi/jupyter-env/bin/python3    # 推荐：显式、不依赖 shell 状态
$PY main.py nine_grid

# 或者先激活 venv
source /home/pi/jupyter-env/bin/activate
python main.py nine_grid
```

4. Python 文件的执行方式（按用途分四类）

先决条件： `cd /home/pi/Robot_Competition`

所有入口都以仓库根为基准，`core.paths.RESULT_DIR` 也是相对它计算的。

① 关卡入口 `main.py`

```
cd /home/pi/Robot_Competition
/home/pi/jupyter-env/bin/python3 main.py nine_grid
/home/pi/jupyter-env/bin/python3 main.py goodluck
/home/pi/jupyter-env/bin/python3 main.py stairs_hurdle
```

`main.py` 的 `LEVELS` 表只有这三个键。`line_seeker_tracking` 与 `apriltag_sorting_task` 不在其中，那两个要直接跑文件（见 ③）。

② `tools/` 下的工具 —— 直接给文件路径

```
/home/pi/jupyter-env/bin/python3 tools/run_layout_scan.py --no-lock --budget 600 --out m5_pos1.json
/home/pi/jupyter-env/bin/python3 tools/field_probe_ninegrid.py
/home/pi/jupyter-env/bin/python3 tools/field_camera_sweep.py --what all
/home/pi/jupyter-env/bin/python3 tools/diag_arrive.py
```

这些脚本自己做了 `sys.path.insert` 把仓库根加进去，从仓库根跑即可。

③ `levels/` 下未注册的关卡 —— 同样直接跑文件

```
/home/pi/jupyter-env/bin/python3 levels/line_seeker_tracking.py
/home/pi/jupyter-env/bin/python3 levels/apriltag_sorting_task.py
```

`line_seeker_tracking.py` 自己会 `sys.path.insert` 到 `vision/`（它 `from line_detector import ...`），不需要额外操心路径。

④ ❌ 不要用 `python -m`

```
/home/pi/jupyter-env/bin/python3 -m tests.test_nine_grid_sim # ❌ 必失败
```

两个原因：

- ① `tests/` 没有 `__init__.py`，不是包；
- ② 更坑的是 `site-packages` 里有个同名 `tests` 包会遮蔽本仓库的 `tests/`，报 `No module named tests.xxx`。

PC 上同理，一律用文件路径：`python tests\test_nine_grid_sim.py`

5. shell 脚本的执行方式

```
cd /home/pi/Robot_Competition
bash debug.sh          # 启动 debug 镜像 → http://192.168.31.209:8081
bash debug.sh stop     # 停止
bash debug.sh status   # 查看状态
bash debug.sh log      # 查看日志 (tail /tmp/debug_server.log)
```

```
tools/train/export_onnx.sh    # 训练相关, 需 bash
tools/train/train_baseline.sh
```

- ⚠ 别用 `sh`。
- ⚠ 行尾必须是 `LF` (同步工具已自动转换)。
- ⓘ debug 镜像服务**不占用摄像头**: 全文件无 `cv2.VideoCapture`, 唯一碰设备处是网页按钮触发的 `fswebcam` 子进程 (拍完即退); 后台 watchdog 每 0.5 秒只扫盘分析已有 JPEG。测量期间可以开着, 只需别点网页上的拍照按钮。

6. PC 侧远程操作 (不用登机器人)

```
# 通用: 在机器人上跑脚本 (现场推荐用这个包装器)
python tools/_tmp_field_session.py --check
python tools/_tmp_field_session.py --script _tmp_measure_motion.py --args "--group 1a --run"

# 底层: 直接执行 shell / Python 代码
python tools/exec_on_robot.py --cmd "iw dev wlan0 get power_save"
python tools/exec_on_robot.py --code "print(1+1)"

# 传文件
python tools/sync_to_robot.py --check
python tools/pull_from_robot.py --remote "archive/result/measure_frames" --dest "tests/fixtures/field_
```

PC 侧每个新终端都要先设一次:

```
$env:PYTHONIOENCODING='utf-8'
```

原因: PowerShell 自报 `OutputEncoding = utf-8`, 但 `chcp` 仍是 **936 (GBK)**, Python 按代码页决定 stdout 编码。机器人输出里的 `⇒ / ✓` 会让 `exec_on_robot.py` 在回显阶段抛 `UnicodeEncodeError` —— 机器人其实已经执行完, 只是输出丢了, 容易误判成失败。

7. 速查卡

```
# ===== 机器人侧 =====
cd /home/pi/Robot_Competition
PY=/home/pi/jupyter-env/bin/python3

$PY main.py nine_grid # 跑关卡
$PY tools/run_layout_scan.py --no-lock --out m5_pos1.json # 跑工具（给路径，不要 -m）
bash debug.sh # 跑 shell（不要 sh）

# ===== PC 侧 =====
$env:PYTHONIOENCODING='utf-8' # 每个新终端先设一次
python tools/_tmp_field_session.py --check # 远程跑机器人脚本
python tests\test_nine_grid_sim.py # 跑测试（给路径，不要 -m）
```

8. 故障排查：机器人连不上

第一步永远是查 PC 的 WiFi。

必须连 **eelab901b** 或 **eelab901b_2**；不能连 **Tsinghua-Secure**（校园网与机器人不在同一广播域，必然连不上）。

```
netsh wlan show interfaces # PC：看当前 SSID
ping -n 4 192.168.31.209 # PC：通不通
Test-NetConnection 192.168.31.209 -Port 8888 # PC：Jupyter 端口
arp -a | Select-String "192.168.31" # PC：有没有 ARP 条目
```

症状	结论
PC 侧 SSID 不是 eelab901b*	先改 WiFi，让其主人手动修正
无法访问目标主机 、ARP 无条目	设备不在同一广播域（连错网 / 机器人连了副路由）
ping 时好时坏、丢包	机器人 WiFi 省电模式（已关并持久化，换镜像后需重做）
一直不通	机器人没开机（先看灯 / uptime ）

生成方式：HTML + Edge 无头打印（**--headless --print-to-pdf**），中文字体为系统「微软雅黑」。

源文件：[docs/机器人终端与执行说明.html](#) | 如需更新，改源文件后重新打印即可。